

1 Commandes

Votre programme client-serveur doit suivre un protocole d'échanges de messages sous forme de texte. Vous devez respecter la syntaxe exacte, afin que votre client puisse se connecter à n'importe quel serveur (celui d'un autre groupe par exemple), et que votre serveur puisse être utilisé par n'importe quel client.

Ces commandes sont les échanges entre le client et le serveur, pas les commandes que le joueur doit entrer.

Le plateau du jeu d'échecs a 8 lignes, identifié par un chiffre de 1 à 8, et 8 colonnes identifiées par les lettres de a à h.

1.1 Client vers Serveur

Ces commandes sont celles envoyées par le client. Le serveur envoie systématiquement une réponse, indiquant si la commande a été acceptée, ou s'il y a eu une erreur.

- **register** *nomJoueur motDePasse* : enregistre un nouveau joueur avec son mot de passe. Le *nomJoueur* doit être une chaîne de caractères sans espace, avec au minimum 3 caractères, et 10 caractères au maximum. Le serveur répond OK si le joueur est enregistré, **ERR message** si le nom du joueur n'est pas valide (le joueur est déjà présent par exemple, ou si le nom n'est pas précisé). Le mot de passe doit avoir au moins 6 caractères.
- **connect** *nomJoueur motDePasse* : demande de connexion du client avec son nom de joueur et son mot de passe. Le serveur commence la recherche d'un joueur disponible. Le serveur répond OK si le joueur peut se connecter, **ERR message** si le nom du joueur n'est pas valide (le joueur est déjà présent par exemple, ou si le nom n'est pas précisé). Le serveur commence la recherche d'un joueur disponible.
- **play** *caseSrc caseDst* : pendant une partie active, le joueur déplace sa pièce des coordonnées source aux coordonnées de destination. Le coup est de la forme *a3 a4* pour déplacer la pièce de la case *a3* à *a4*. Le serveur répond OK si le coup est valide, **ERR message** sinon.
- **leave** : le client abandonne la partie, commande valable uniquement pendant une partie active. Le joueur est considéré comme perdant. Le serveur répond OK si l'abandon est accepté, et **ERR message** en cas de problème.
- **quit** : le client quitte, et ferme la connexion avec le serveur. Si une partie est active, le joueur est considéré comme perdant. Le serveur répond OK si la déconnexion est acceptée, et **ERR message** en cas de problème.
- **replay** A la fin d'une partie relance le jeu contre le même adversaire. Les deux joueurs doivent avoir envoyé cette commande.
- **new** A la fin d'une partie
- **sync** *key* : envoie de la clé de chiffrement. Le serveur répond OK
- **promote** *caseSrc newPiece* : promotion du pion de la case *caseSrc* lorsqu'il arrive en dernière ligne. *newPiece* est égale à **q**, **r**, **b**, **k** pour respectivement la reine, la tour, le fou et le chevalier. Le serveur répond OK ou **ERR**.

1.2 Serveur vers Client

Le serveur peut envoyer des commandes lorsqu'il détecte la fin d'une partie

- **start** *[w|b]* : la partie commence en indiquant la couleur du joueur
- **play_ad** *caseSrc caseDst* : coup joué par l'adversaire
- **win** : le joueur a gagné
- **lose** : le joueur a perdu
- **draw** : égalité, pat
- **exit** : fin non prévue d'une partie (en cas de problème côté serveur)
- **sync** *key* : envoi de la clé de chiffrement.

En cas de réception d'une commande non prévue, le serveur répond avec **ERR message**

2 Commandes liées aux fonctionnalités supplémentaires

Pour la suite, on suppose que chaque partie est associée aux informations suivantes :

- un identifiant unique
- le pseudo des deux joueurs
- l'attribution des couleurs
- la date de la partie
- le résultat final pour une partie terminée, le joueur au trait si la partie est en cours.
- la liste des coups joués

Des fonctionnalités supplémentaires sont possibles, si vous décidez de les implémenter, vous devez suivre les commandes suivantes :

2.1 Client vers serveur

- **ask** *nomJoueurAdverse* : demande de nouvelle partie contre un joueur. Le serveur répond OK si la demande est valide, **ERR message** si le nom du joueur adverse n'est pas valide (pas présent dans les joueurs en attente par exemple)
- **accept** *nomJoueurAdverse* : accepte la demande d'un joueur. Le serveur répond OK si la demande est valide, **ERR message** sinon.
- **players** : demande la liste des joueurs disponibles. Le serveur répond OK suivi de la liste des joueurs, **ERR message** sinon.
- **save** : sauvegarde la partie. Le serveur répond OK en cas de succès, **ERR message** sinon.
- **list_games** : permet de récupérer une liste des parties jouées par le joueur. Par défaut, on récupère uniquement les métadonnées de la partie ainsi que son identifiant.
- **load** *idPartie* : permet de récupérer l'intégralité d'une partie. Le serveur répond OK en cas de succès, **ERR message** sinon.